

Extending Google Test for Statistical Benchmarking, CUDA Profiling, and CI Performance Regression

KEVIN GOMEZ

The Problem

The problem, in one slide

Performance work in C++ lives outside the test suite.

- ▶ **Timing usually stops at the basics.** A loop around the code gives wall time and CPU time. It can't say whether that number is stable, or why it is what it is
- ▶ **Benchmarks live apart from the tests.** A benchmark framework builds its own executable, with its own `main`, its own command line and its own printed report. The test suite, and the build that runs it, never see the result
- ▶ **Every profiler is a separate workflow.** Each has its own command line, its own output format and its own permissions, so each is learned and set up on its own
- ▶ **GPU code adds another set of tools,** run a different way again
- ▶ **CI runs every unit test and not one benchmark.** Nothing fails when the code gets slower, so the people who find the regression are the users

The basics: the timing loop

Call the function N times, read a clock before and after, divide by N. There are two clocks worth reading:

```
using namespace std::chrono;
auto wall0 = steady_clock::now();           // wall clock: what a stopwatch reads
auto cpu0  = std::clock();                  // CPU clock: time this process spent running
for (int i = 0; i < N; ++i) functionUnderTest();
double wallUs = duration<double, std::micro>(steady_clock::now() - wall0).count() / N;
double cpuUs  = 1e6 * (std::clock() - cpu0) / CLOCKS_PER_SEC / N;
std::cout << wallUs << " us wall, " << cpuUs << " us cpu, per call\n";    // ...and now what?
```

- ▶ **Wall time** is elapsed real time: your code, any waiting (I/O, a lock, a GPU the call waits for), and other processes on the core.
- ▶ **CPU time** is time the processor spent running your code. Waiting doesn't count. With four threads it can exceed wall time.
- ▶ Both are **average time per call**, which is latency. Flip either over and it's **throughput**, calls per second.
- ▶ One number, one run, one machine. No warmup, no repeats, no spread. Printed, and nothing fails.

A profiler per question

Question	Tool	How you run it	Needs
where are the cycles?	perf	<code>perf record ...</code>	sysctl setting, matching kernel package
where are the allocations?	heaptrack, massif	wrap the binary	valgrind or heaptrack installed
why is it <i>waiting</i> ?	bpftrace, off-CPU	a script, as root	root, tracefs, kernel probes
what is the GPU doing?	nsys, ncu	wrap, then a GUI	driver counter permission
is it correct under threads?	helgrind	wrap, 50× slower	valgrind

Five workflows, five permission models, zero shared output.

CI: unit tests are three lines, benchmarks are five questions

```
# .github/workflows/ci.yml, every project has this
- run: cmake --preset release && cmake --build --preset release
- run: ctest --preset release           # green check, or red X
```

Now write the benchmark job.

- ▶ **Which machine runs the job?** On a hosted VM, one change of VM generation moved every median 14–56% between two runs of identical code
- ▶ **Which baseline?** compared to what, stored where, blessed by whom
- ▶ **What threshold?** 5%? on a 2 μ s operation?
- ▶ **How does it reach the hardware you ship on?** A Jetson on a lab bench isn't on GitHub's network, and whatever the job needs has to be installed on it
- ▶ **What permissions?** profilers want root; CI runners don't have it

The example is GitHub Actions. GitLab CI and Jenkins raise the same five questions.

Existing benchmark frameworks, and what they do well

Framework	Does well	Where it lives
Google Benchmark	repetitions with mean / median / stddev / CV, hardware counters, JSON output, a compare script	its own program (<code>BENCHMARK_MAIN</code>) and runner
nanobench	header-only, quick to add, median and error estimate, perf counters	a function call in any program; a table you read
Catch2 BENCHMARK	microbenchmarks inside Catch2 tests, mean and stddev	beside the tests; reported, not asserted
hyperfine	whole-process timing, warmup, A/B of two commands	the shell

```
$ ./bm --benchmark_repetitions=5          # Google Benchmark, timing a + b on two short strings
BM_AddStrings_mean          5.70 ns      5.70 ns      5
BM_AddStrings_median        5.68 ns      5.68 ns      5
BM_AddStrings_stddev        0.030 ns     0.030 ns     5
BM_AddStrings_cv            0.53 %       0.53 %       5
```

Good measurers, all of them. Real statistics, not one number.

Why vernier

Everything the last slide does well, **inside the test suite you already have**. Then the parts nothing else covers.

	the frameworks on the last slide	vernier
Statistics	Repetitions and aggregates.	The same, plus percentiles, and a stability verdict the test can assert on.
What a benchmark is	A separate program, or a report printed beside the tests.	A Google Test TEST . Same file, same runner, same ctest .
When a result is slow or noisy	It's printed.	The result is a value in the test, so EXPECT_* works on it and the test can fail , the same way a wrong answer does.
In CI	A compare script you run outside the build.	A CLI compares the run to a committed baseline and fails the job.
Finding out why	Hardware counters, if you ask for them.	Any profiler from the earlier table, with one flag, on the same binary.
GPU	No GPU support.	CUDA kernels are measured by the same tests. Nsight runs through the same flag. Per-kernel counters are built in.

What Is Google Test?

Google Test, in thirty seconds

github.com/google/googletest — a widely used C++ test framework, and probably a familiar one. A test is a function; its body checks answers.

Macro	What it gives you
<code>TEST(Suite, Name)</code>	a standalone test
<code>TEST_F(Fixture, Name)</code>	tests that share setup from a fixture class (<code>SetUp()</code> / <code>TearDown()</code>)
<code>TEST_P(Fixture, Name) + INSTANTIATE_TEST_SUITE_P</code>	one test body, run once per parameter value
<code>TYPED_TEST(Fixture, Name)</code>	one test body, run once per type
<code>EXPECT_*</code> / <code>ASSERT_*</code>	the checks; ASSERT stops the test on failure
<code>MOCK_METHOD</code> / <code>EXPECT_CALL</code>	Google Mock: a fake implementation of an interface, with expectations

Also in the box: filtering (`--gtest_filter`), sharding, XML reports, event listeners, `ctest` discovery, and `gtest_main`, a ready-made `main()`.

A function to test

Before any test macro, something to test. The smallest function worth the trouble:

```
template <typename T>
T add(const T& a, const T& b) {
    return a + b;
}
```

Call	Result
<code>add(2, 3)</code>	5
<code>add(2.5, 0.25)</code>	2.75
<code>add(std::string("ab"), std::string("cd"))</code>	"abcd"

The next three slides test this one function, one Google Test macro at a time.

TEST: a plain test

```
TEST(Add, AddsTwoInts) {                                // suite "Add", test "AddsTwoInts"
    EXPECT_EQ(add(2, 3), 5);                             // a check: are the two sides equal?
}

TEST(Add, AddsTwoDoubles) {                             // same suite, second test
    EXPECT_DOUBLE_EQ(add(2.5, 0.25), 2.75);              // floating point: equal within a few ulps
}

TEST(Add, ConcatenatesStrings) {
    EXPECT_EQ(add(std::string("ab"), std::string("cd")), "abcd");
}
```

- ▶ `TEST(Suite, Name)`: a function the runner finds by its two names; no registration, no `main()` to write
- ▶ `EXPECT_*` records a failure and keeps going; `ASSERT_*` records it and **stops this test**
- ▶ On failure the output shows both sides: the expression and what it evaluated to

TEST_F: tests that share a fixture

```
class AddFixture : public ::testing::Test {           // a fixture: setup the tests share
protected:
    void SetUp() override { values = {1, 2, 3, 4}; } // runs before EACH test below
    std::vector<int> values;                          // the tests see this directly
};

TEST_F(AddFixture, SumsTheList) {                    // this test runs inside a fresh AddFixture
    int total = 0;
    for (int v : values) total = add(total, v);
    EXPECT_EQ(total, 10);
}

TEST_F(AddFixture, ListIsUntouchedBetweenTests) {    // SetUp ran again: values is {1, 2, 3, 4}
    EXPECT_EQ(values.size(), 4u);
}
```

- ▶ A **fixture** is a class deriving from `::testing::Test`; `TEST_F` makes the test body a member of it
- ▶ `SetUp()` and `TearDown()` run around **every** test, so tests never share state by accident
- ▶ `SetUpTestSuite()` / `TearDownTestSuite()` exist for setup too expensive to repeat

TEST_P: one body, many values

```
class AddCase : public ::testing::TestWithParam<std::tuple<int, int, int>> {};  
  
TEST_P(AddCase, MatchesExpected) {                               // one test body...  
    const auto [A, B, EXPECTED] = GetParam();                    // ...GetParam() is this run's row  
    EXPECT_EQ(add(A, B), EXPECTED);  
}  
  
INSTANTIATE_TEST_SUITE_P(Table, AddCase,                          // ...run once per row, three tests in the output  
    ::testing::Values(std::make_tuple(0, 0, 0),  
                       std::make_tuple(2, 3, 5),  
                       std::make_tuple(-1, 1, 0)));
```

- ▶ The fixture derives from `TestWithParam<T>`; `GetParam()` hands the body one `T`
- ▶ `INSTANTIATE_TEST_SUITE_P` supplies the values (`Values`, `Range`, `ValuesIn` a container, `Combine`)
- ▶ Each value is its own test: `Table/AddCase.MatchesExpected/0`, `/1`, `/2`, so a failing row is named

Vernier: Performance Tests as Google Test Tests

Vernier, in thirty seconds

A C++23 library that sits **on top of** Google Test. A performance test is a **TEST** with a measurement object inside it. There is no second runner.

Piece	What it is
<code>PERF_TEST</code> , <code>PERF_TEST_F</code> , <code>PERF_TEST_P</code>	<code>TEST</code> , <code>TEST_F</code> , <code>TEST_P</code> under another name, so a reader can tell a performance test from a unit test. Same runner, same filters, same fixtures
<code>PERF_GUARD(perf)</code>	the extension : creates <code>perf</code> , the measurement object for this test
<code>perf.throughputLoop(fn)</code> , <code>perf.contentionRun(fn)</code> , <code>perf.measured(fn)</code>	the experiments: time <code>fn</code> in a loop, across threads, or as a loop you wrote
<code>EXPECT_*</code> on the result	the check: calls per second, spread, stability. A bad number fails the test
<code>PERF_MAIN()</code>	the <code>main()</code> : parses <code>--cycles</code> , <code>--repeats</code> , <code>--csv</code> , <code>--profile</code> , then <code>RUN_ALL_TESTS()</code>
<code>--profile <tool></code>	the profiler hook: one flag, any of 14 profilers, on this same binary
<code>bench</code>	a companion CLI: run, compare against a baseline, diagnose the machine

CPU backends

<code>--profile</code>	The question it answers
<code>perf</code>	where are the cycles? Sampled hot spots and hardware counters (cache misses, branch misses)
<code>gperf</code>	which function eats the time, as a flame graph
<code>callgrind</code>	exact instruction counts, identical every run: ideal for A/B comparisons
<code>heaptrack</code>	every allocation with its call stack, ranked by site
<code>massif</code>	heap size over time: where memory peaks, and who owns it
<code>jemalloc</code>	sampled allocation hot spots at very low overhead
<code>memcheck</code>	leaks, use-after-free, uninitialized reads
<code>helgrind</code>	data races and lock-ordering bugs in threaded code
<code>offcpu</code>	where threads block and wait: the time the others can't see
<code>bpftrace</code>	kernel tracing: syscalls, scheduler events, your own probe scripts
<code>rapl</code>	energy per test, on Intel

Eleven tools, one flag. Same binary, same test, a different question each time.

GPU backends

Tool	The question it answers
<code>nsight</code> (Nsight Systems)	what happened when: a timeline of kernels, memory copies, and CPU/GPU overlap
<code>ncu</code> (Nsight Compute)	what a kernel is bound by: memory vs compute throughput, occupancy
<code>compute-sanitizer</code>	is device code correct: out-of-bounds access, races, uninitialized memory
GPU kernel tests (built in, no flag)	time per kernel launch, measured on the GPU with CUDA events, plus CUPTI's launch count and register use per kernel
BENCH_NVTX_SCOPE (built in)	named ranges, so Nsight timelines show your phases

The first three use the same `--profile` flag and the same test binary as the CPU tools: fourteen profilers behind one flag.

`ncu` needs root on Jetson. Inside a container, `nsight` prints the command to run instead of attaching, so profile GPUs on the host.

Step one: add it to the build

```
include(FetchContent)
FetchContent_Declare(googletest GIT_REPOSITORY https://github.com/google/googletest.git
                        GIT_TAG v1.15.2) # your gtest, declared first: you own the version
FetchContent_Declare(vernier GIT_REPOSITORY https://github.com/apexedgesystems/vernier.git
                        GIT_TAG v1.0.4)
FetchContent_MakeAvailable(googletest vernier)

add_executable(Add_PTEST ptst/Add_pTest.cpp)
target_link_libraries(Add_PTEST PRIVATE vernier::bench GTest::gtest) # the library, beside gtest
add_test(NAME Add_PTEST COMMAND Add_PTEST --cycles 100 --repeats 5) # registered like any other test
```

- ▶ Two **FetchContent** blocks. Google Test is declared first, so the consumer owns its version and vernier rides along
- ▶ One link line: **vernier::bench** next to **GTest::gtest**
- ▶ One **add_test** line: **ctest** runs the performance tests in the same suite as the unit tests, with a quick cycle count

The same add, as a performance test

```
PERF_TEST(Add, StringsThroughput) {  
    PERF_GUARD(perf);  
    const std::string A = "ab", B = "cd";  
    perf.warmup([&] { add(A, B); });  
    auto r = perf.throughputLoop([&] { add(A, B); });  
    EXPECT_GT(r.callsPerSecond, 1e6);  
}  
PERF_MAIN()  
  
// 1. PERF_TEST is TEST: suite "Add", test "StringsThroughput"  
// 2. the measurement object, named after this test  
// 3. one untimed call first, so caches and the allocator are warm  
// 4. times --cycles calls per round, --repeats rounds; r has the stats  
// 5. an ordinary gtest check, on a measured number  
// 6. main(): flags, a result listener, RUN_ALL_TESTS()
```

```
$ ./Add_PTEST --cycles 10000 --repeats 5  
[ RUN      ] Add.StringsThroughput  
[Add.StringsThroughput] 0.008 us/call CV=3.0% ~126.6M calls/s (p10=0.008 p90=0.008 sd=0.000)  
[ OK      ] Add.StringsThroughput (3 ms)
```

- ▶ Lines 1, 5 and 6 are Google Test. Lines 2 to 4 are vernier. `fn` is the code under test, as a lambda, same as before
- ▶ The output is the normal gtest output plus one result line per test

The perf object

`PERF_GUARD(perf)` creates a `PerfCase` named after the test. Its methods are the experiments. `fn` is the code under test: any callable that takes no arguments, usually a lambda such as `[&] { add(a, b); }`.

Call	What it does
<code>perf.warmup(fn)</code>	runs <code>fn</code> untimed first: once by default, or a count vernier picks with <code>--warmup 0</code>
<code>perf.throughputLoop(fn)</code>	calls <code>fn</code> back to back, <code>--cycles</code> times per round, on one thread. One sample per round: its wall time $\div$ cycles
<code>perf.contentionRun(fn)</code>	starts <code>--threads</code> threads at the same moment, each calling <code>fn</code> <code>--cycles</code> times. One sample per round: wall time $\div$ (threads $\times$ cycles)
<code>perf.measured(fn)</code>	times a loop you wrote yourself, once per round, $\div$ cycles. The other two are built on it

Each runs `--repeats` rounds and returns a `PerfResult`: **median, p10, p90, p99, min, max, mean, standard deviation, CV, calls per second**, and a **stability flag**. It prints the result line and hands the row to the CSV listener.

The method chooses the experiment: the same `fn` under `throughputLoop` and under `contentionRun` answers two different questions.

Reading the result

```
[Add.StringsThroughput] 0.008 us/call CV=3.0% ~126.6M calls/s (p10=0.008 p90=0.008 sd=0.000)
```

Piece	What it means
0.008 us/call	the median of the round averages: the time for one call
CV=3.0%	coefficient of variation, standard deviation ÷ mean: how much the rounds disagreed
[UNSTABLE] / OK	the CV is over its limit (20% for the smallest operations, down to 3% for 1 KiB and up). In the test, r.stable is false
~126.6M calls/s	the same median, as a rate
p10 / p90	the 10th and 90th percentile across the rounds
sd	the standard deviation of the rounds, in microseconds

At the end of the run, a summary table; with `--csv`, one row per test with every statistic, plus the cycle and repeat counts, timestamp, git hash, host and platform:

Test	Median (us)	CV%	Calls/s	Status
Sizes/AddSweep.StringLength/0	0.010	0.4%	103.1M	OK
Sizes/AddSweep.StringLength/2	0.120	34.4%	8.3M	UNSTABLE

A slow result fails the test

1. The test asserts on the result: a time budget, and that the measurement can be trusted:

```
auto r = perf.throughputLoop([&] { add(A, B); });           // A and B: two 4 KB strings
EXPECT_TRUE(r.stable);                                     // did the rounds agree?
EXPECT_LT(r.stats.median, 0.150) << "add() on 4 KB strings is over its 0.15 us budget";
```

```
[Add.LongStringsBudget] 0.126 us/call CV=1.0% ~7.9M calls/s (p10=0.124 p90=0.127 sd=0.001)
```

2. A tidy-looking refactor: `add(const T& a, const T& b)` becomes `add(T a, T b)`. **Every call now copies both strings:**

```
[Add.LongStringsBudget] 0.211 us/call CV=0.6% ~4.8M calls/s (p10=0.209 p90=0.212 sd=0.001)
Add_pTest.cpp:18: Failure
Expected: (r.stats.median) < (0.150), actual: 0.210525 vs 0.15
add() on 4 KB strings is over its 0.15 us budget
[ FAILED ] Add.LongStringsBudget (43 ms)
```

3. The unit tests still pass, because the answers are still right. The performance test is what caught it.

Fixtures and parameters, unchanged

```
class AddSweep : public ::testing::TestWithParam<std::size_t> {};    // the same fixture as TEST_P

PERF_TEST_P(AddSweep, StringLength) {                                // PERF_TEST_P is TEST_P
    PERF_GUARD(perf);
    const std::string A(GetParam(), 'a'), B(GetParam(), 'b');        // GetParam(): this run's string length
    perf.warmup([&] { add(A, B); });
    auto r = perf.throughputLoop([&] { add(A, B); });
    EXPECT_TRUE(r.stable);
}
INstantiateTestSuiteP(Sizes, AddSweep, ::testing::Values(2, 64, 4096)); // three lengths, three tests
```

```
[Sizes/AddSweep.StringLength/0] 0.010 us/call CV=0.4% ~103.1M calls/s (p10=0.010 p90=0.010 sd=0.000)
[Sizes/AddSweep.StringLength/1] 0.039 us/call CV=2.0% ~25.4M calls/s (p10=0.039 p90=0.041 sd=0.001)
[Sizes/AddSweep.StringLength/2] 0.120 us/call CV=34.4% ~8.3M calls/s (p10=0.118 p90=0.193 sd=0.049) [UNSTABLE]
```

- ▶ **Nothing here is new to Google Test.** The fixture, `GetParam()` and `INstantiateTestSuiteP` are the ones from the `TEST_P` slide. Vernier adds the `PERF_` prefix and the guard
- ▶ A sweep over sizes is a parameterized test; a scene with shared setup is a fixture (`PERF_TEST_F`)
- ▶ `--gtest_filter`, sharding, `ctest` discovery, listeners: all still yours

Running it: the flags

A vernier binary is an ordinary Google Test binary. `PERF_MAIN()` adds these flags; gtest's own (`--gtest_filter`, `--gtest_list_tests`) still work.

```
./Add_PTEST --cycles 10000 --repeats 5 --csv add.csv
./Add_PTEST --target-time 60ms --repeats 10 --gtest_filter='Sizes/*'
./Add_PTEST --profile heaptrack --gtest_filter=Add.StringsThroughput
```

Flag	What it controls	Default
<code>--cycles N</code>	calls per round	10,000
<code>--repeats N</code>	rounds per test; each round is one sample	10
<code>--target-time 60ms</code>	picks cycles so each round lasts about this long (us, ms, s)	off
<code>--warmup N</code>	untimed calls before timing; 0 lets vernier choose	1
<code>--threads N</code>	threads for <code>contentionRun</code>	1
<code>--msg-bytes N</code>	payload size, which sets the CV threshold	64
<code>--csv FILE</code>	one row per test, for CI	off
<code>--profile TOOL</code>	attaches a profiler to every test in the run	off
<code>--quick</code>	fewer cycles and repeats, relaxed threshold, for iterating	off

One flag, every profiler: how it works

`PERF_GUARD` builds the `perf` object with the profiler named by `--profile` attached. The profiler gets two calls, one on either side of the measured loop:

```
PerfCase perf{testName, cfg};
auto prof = Profiler::make(cfg, testName);           // looks up --profile's backend in a registry
perf.setBeforeMeasureHook([&](auto&) { prof->beforeMeasure(); }); // e.g. start sampling
perf.setAfterMeasureHook([&](auto&, const Stats& s) { prof->afterMeasure(s); }); // e.g. stop, write the artifact
```

Two kinds of backend. **In-process** ones (`perf`, `gperf`, `bpftime`, `offcpu`, `rapl`) start and stop inside those hooks. **Wrapper** ones (`valgrind`'s `callgrind`, `memcheck`, `massif`, `helgrind`; `heaptrack`; `Nsight`) have to launch the process, so `bench run` launches it and the binary notices:

```
$ bench run Add_PTEST --profile heaptrack -- --gtest_filter=Add.LongStringsBudget --cycles 10000 --repeats 1
Running: heaptrack -o ../Add_PTEST.heaptrack/run Add_PTEST --profile heaptrack ...
[heaptrack] wrapping detected; heap profile will be written at process exit.
[Add.LongStringsBudget] 25.523 us/call CV=0.0% ~39.2K calls/s (p10=25.523 p90=25.523 sd=0.000)
heaptrack stats: allocations: 20306 leaked allocations: 33 temporary allocations: 25
```

Every backend writes into its own folder under `--profile-output-dir: <Suite.Test>.<tool>/` for the in-process ones, `<binary>.<tool>/` for the wrapped ones.

The doctor: what works on this machine

```
$ bench doctor Add_PTEST
=== Profiler Backend Doctor ===
[WARN] bpftrace    bpftrace available but not running as root
                  Run with sudo, or set BENCH_SUDO=1 with a scoped sudoers grant (bpftrace + kill).
[FAIL] heaptrack   heaptrack binary not found on PATH
                  apt install heaptrack (and optionally heaptrack-gui).
[FAIL] perf        perf_event_paranoid=4 (all perf access blocked)
                  Lower with: sudo sysctl -w kernel.perf_event_paranoid=1 (resets on reboot; rerun each session).
[FAIL] rapl        RAPL MSRs present but not readable by current user
                  Re-run with sudo, or `sudo setcap cap_sys_rawio+ep <binary>`.
12 backend(s), 10 fail.
```

Privilege	Tools
nothing, ever	the whole CI gate; valgrind's four; heaptrack; gperf; <code>bench compare</code>
a <code>sysctl (perf_event_paranoid)</code>	<code>perf</code>
root, or a scoped <code>BENCH_SUDO</code>	<code>bpftrace</code> , <code>offcpu</code> (two sudoers lines)
root on Jetson	<code>ncu</code> ; <code>nsight</code> and CUPTI stay unprivileged

The bench CLI

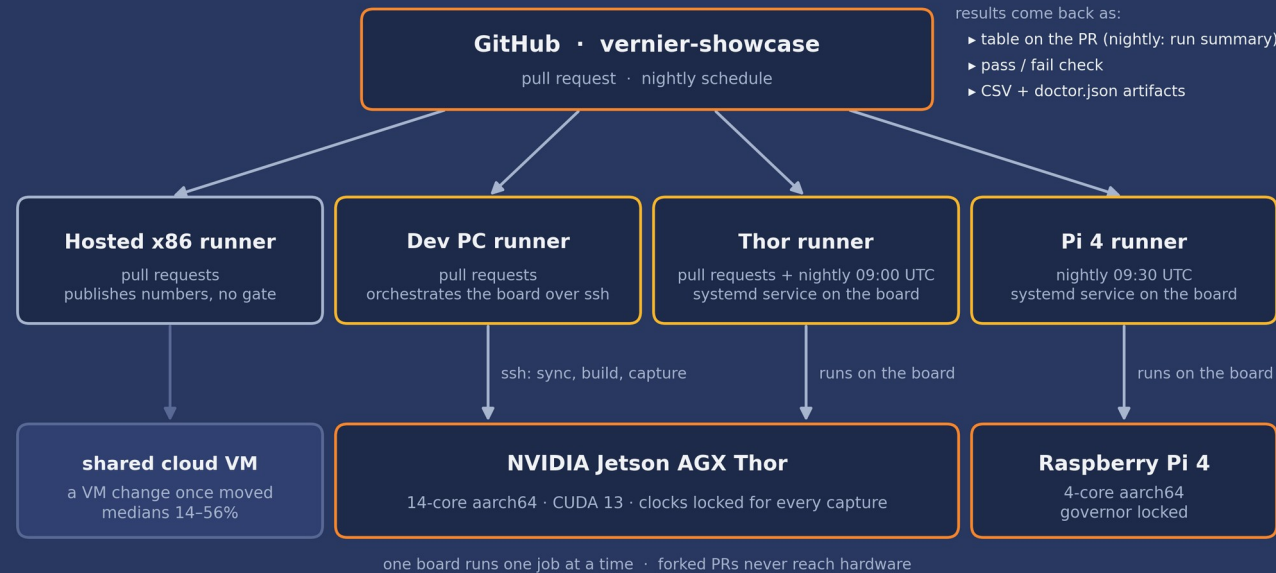
The test binary measures on its own. **bench** runs it more conveniently and judges the results:

```
bench run Add_PTEST --profile heaptrack -- --gtest_filter=Add.StringsThroughput
bench compare baseline.csv candidate.csv --fail-on-regression
bench doctor Add_PTEST
```

Command	What it does
<code>bench run BINARY [options] -- [test args]</code>	finds the binary by short name, forwards vernier's flags, and launches the wrapper profilers (valgrind family, heaptrack, Nsight). <code>--taskset</code> pins it to CPUs
<code>bench compare BASELINE CANDIDATE</code>	compares two CSVs test by test. A row is a regression when it's slower by more than 5% and the difference is statistically significant
<code>bench compare ... --markdown</code>	the table CI posts on a pull request
<code>bench compare ... --fail-on-regression</code>	exits with an error on any regression, which fails the CI job
<code>bench doctor BINARY</code>	the previous slide

And How Does This Fit Into CI?

One figure: the fleet



- ▶ Four runners, two shapes: a hosted VM that publishes but never gates, a lab PC that drives a board over ssh, and a runner living on each board as a service
- ▶ Every lane ends the same way: a table, a pass/fail check, and the CSV and `doctor.json` as artifacts
- ▶ Each board is compared against **its own baseline**: a CSV captured on that board and committed to the repository. A person refreshes it, in the pull request that earned the change

What one lane does

Trimmed from the Thor lane's workflow. Build, then five steps:

```
- name: Record the board's capability manifest          # what works here; never fails the lane
  run: BENCH_SUDO=1 bench doctor GridFilter_PTEST --json > doctor.json

- name: Capture under locked clocks                    # jetson_clocks, run every binary with --csv, restore
  run: bash .github/scripts/thor-capture.sh candidate-out

- name: Compare against the committed baseline
  run: |
    bench compare bench/baselines/thor/baseline.csv candidate.csv --markdown > report.md # the table
    cat report.md >> "$GITHUB_STEP_SUMMARY"
    bench compare bench/baselines/thor/baseline.csv candidate.csv --fail-on-regression      # the gate

- name: Upload candidate, contention exhibit and doctor.json # always, even on failure
  uses: actions/upload-artifact@v4

- name: Post sticky PR comment                          # one comment per lane, updated in place
  run: gh api "repos/$REPO/issues/$PR/comments" -F body=@report.md
```

What had to be set up on the board: a way for CI to reach it. Here that is a GitHub runner installed on the Thor as a service; the other Thor lane keeps the runner on a PC beside it and drives the board over ssh. **Vernier itself needs nothing installed:** the measurement engine is compiled into the test binary, and **bench** is one executable built with the project. Profilers are an escalation, set up once per board when you want the "why".

What it looks like on a pull request

github-actions Bot commented [last week](#)

Benchmark comparison (thor)

Test	Baseline	Candidate	Delta	%	Result
Analyzer.V0BatchThroughput	1364.94000	1361.80000	-3.14000	-0.2%	[] neutral
Analyzer.V0SingleLineLatency	0.80000	0.79000	-0.01000	-1.3%	[] neutral
Analyzer.V1BatchThroughput	625.79000	795.10000	+169.31000	+27.1%	[!] REGRESSION
Analyzer.V1SingleLineLatency	0.67000	0.69000	+0.02000	+3.0%	[] neutral
Analyzer.V3BatchThroughput	210.02000	207.03000	-2.99000	-1.4%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_2	1.65750	1.67250	+0.01500	+0.9%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_2048	148.36100	202.80500	+54.44400	+36.7%	[!] REGRESSION
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_32	11.65620	12.11000	+0.45380	+3.9%	[] neutral
Sizes/PayloadSweep.BatchThroughputScalesStably/lines_8	5.86625	6.01250	+0.14625	+2.5%	[] neutral

A refactor that changed no answers. The Thor lane flagged **two rows**, the check went **red**, and the hosted x86 lane passed because it doesn't gate.

Vernier at Work: Two Simple Examples

Simple example (CPU): the baseline function

The function under test joins strings, with a separator after each one:

```
std::string joinV0(const std::vector<std::string>& parts, char sep) {  
    std::string out;  
    for (const std::string& p : parts)  
        out = out + p + sep;           // reads naturally; what does it cost?  
    return out;  
}  
  
joinV0({"alpha", "beta", "gamma"}, ','); // returns "alpha,beta,gamma,"
```

- ▶ It is correct: a unit test checks that answer, the empty list, and a single string
- ▶ What is being joined doesn't matter here. The question is what one call costs, and where that cost comes from

Simple example (CPU): the vernier test, and the commands

```
class Join : public ::testing::TestWithParam<std::size_t> {}; // the parameter: how many strings

PERF_TEST_P(Join, V0) {
    PERF_GUARD(perf);
    const auto PARTS = makeParts(GetParam(), 42); // same input every run, built outside the timing
    perf.warmup([&] { (void)joinV0(PARTS, ','); });
    const auto R = perf.throughputLoop([&] { (void)joinV0(PARTS, ','); }, "join_v0");
    EXPECT_GT(R.callsPerSecond, 1.0); // the gate. R also has R.stable, R.stats.median, R.stats.p99 ...
    // EXPECT_TRUE(R.stable); // e.g. fail when the rounds disagree too much
}

INstantiateTestSuite_P(Parts, Join, ::testing::Values(10, 100, 1000, 10000, 100000), byCount);
```

One test binary, then one flag per tool:

```
./Join_PTEST --target-time 60ms --repeats 10 --csv sweep.csv # time per call, every size
bench run Join_PTEST --profile gperf -- --gtest_filter='*V0/n1000' # where the time goes, by function
bench run Join_PTEST --profile callgrind -- --gtest_filter='*V0/n1000' # instructions, exactly
bench run Join_PTEST --profile heaptrack -- --gtest_filter='*V0/n1000' # every allocation
bench run Join_PTEST --profile massif -- --gtest_filter='*V0/n1000' # peak heap
bench run Join_PTEST --profile memcheck -- --gtest_filter='*V0/n1000' # memory errors and leaks
```

Simple example (CPU): the raw output

What those commands print for `joinV0` at 1,000 strings, trimmed to the lines that matter. The next slide reads them.

```
the vernier test
[Parts/Join.V0/n1000] 105.775 us/call CV=0.7% ~9.5K calls/s (p10=105.307 p90=106.404 sd=0.715)

--profile gperf                                samples, by function
Total: 319 samples
    153 48.0% 48.0%    153 48.0% __memcpy_avx_unaligned_erms
     62 19.4% 67.4%     68 21.3% _int_malloc
     33 10.3% 77.7%     41 12.9% _int_free

--profile callgrind                            instructions, whole run of about 100 calls
197,153,716 (100.0%) PROGRAM TOTALS
 78,599,100 (39.87%) memmove-vec-unaligned-erms.S:__memcpy_avx_unaligned_erms
59,829,084 (30.35%) malloc.c:malloc

--profile heaptrack                            whole run of about 100 calls
calls to allocation functions: 201889 (611784/s)
peak heap memory consumption: 157.05K

--profile memcheck
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Simple example (CPU): the baseline results

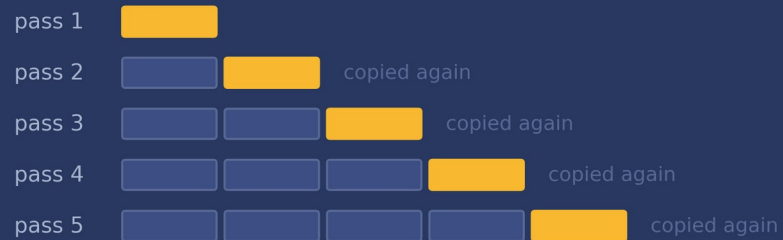
Joining 1,000 short strings:

Reading	Tool	joinV0
Time per call	the vernier test	105.7 μs , rounds agree to 1%
Where the time is	--profile gperf	48% memcpy, 21% malloc, 13% free. Almost none in joinV0 itself
Instructions per call	--profile callgrind	1.91 million: 40% copying, 30% allocating
Allocations per call	--profile heaptrack	1,995 : two for every string
Peak heap	--profile massif	157 KB
Memory errors, leaks	--profile memcheck	none

What it tells us: the function isn't slow at *joining*. It's slow at copying and allocating: each pass builds two temporary strings and copies everything joined so far.

Simple example (CPU): the optimization

V0: every pass builds a new string



Each pass copies everything joined so far,
then adds one new piece.

V1: one buffer, the right size



One buffer is sized up front,
and each piece is written into it once.

```
std::string joinV1(const std::vector<std::string>& parts, char sep) {  
    std::size_t total = 0;  
    for (const std::string& p : parts) total += p.size() + 1;    // measure once  
    std::string out;  
    out.reserve(total);                                           // one allocation, the right size  
    for (const std::string& p : parts) { out += p; out += sep; } // append in place: no temporaries  
    return out;  
}
```

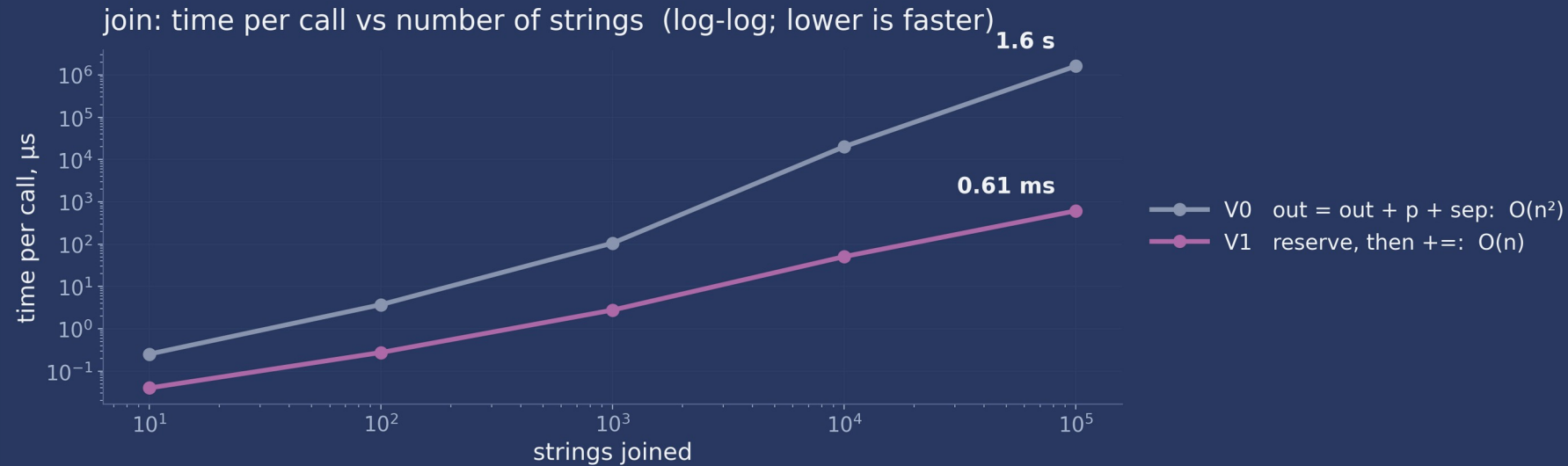
Same output, character for character; a unit test holds `joinV1` to `joinV0` from zero strings to 10,000.

Simple example (CPU): the new results

Reading (1,000 strings)	Tool	joinV0	joinV1	
Time per call	the vernier test	105.7 μ s	2.76 μs	38× faster
Where the time is	--profile gperf	82% in memcpy, malloc, free	99.6% inside joinV1	the time moved into our code
Instructions per call	--profile callgrind	1,910,000	59,000	32× fewer
Allocations per call	--profile heaptrack	1,995	1	exactly what reserve promised
Peak heap	--profile massif	157 KB	127 KB	no temporaries
Memory errors, leaks	--profile memcheck	none	none	still correct

Every tool agrees, and each one proves a different part of the claim.

Simple example (CPU): the new results, across sizes



- ▶ The test is parameterized over the number of strings, so the same run gives the whole curve
- ▶ **V0 is quadratic, $O(n^2)$:** ten times the strings, a hundred times the work. At 100,000 strings it takes **1.6 seconds**
- ▶ **V1 is linear, $O(n)$:** ten times the strings, ten times the work. 0.61 ms at 100,000 strings, **2,600× faster**. At 1,000 strings it was "only" 38×

Simple example (GPU): the job, and why a GPU

Take two long arrays, **x** and **y**. Scale every **x** by a constant, and add it into **y**. It is the inner step of a lot of numerical code, and the classic first GPU program (its library name is SAXPY).

```
void saxpyCpu(float a, const std::vector<float>& x, std::vector<float>& y) {  
    for (std::size_t i = 0; i < x.size(); ++i)  
        y[i] = a * x[i] + y[i];  
}
```

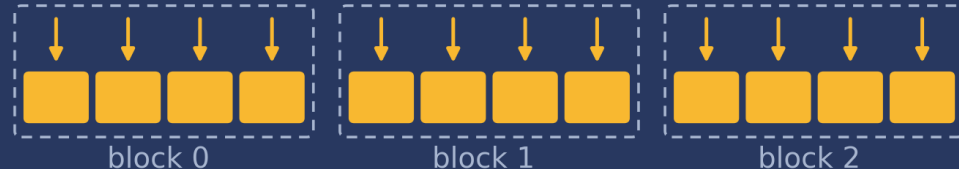
// element i never looks at any other element

CPU: one loop, one element after another



The loop reaches element *i* only after it has finished the ones before it.

GPU: one thread per element, all at once



No element needs another element's answer, so every element can have its own thread.

Simple example (GPU): the baseline function

Two pieces, because there are two processors. The **kernel** is the code the GPU runs. The **host function** is ordinary C++ on the CPU: it moves the data over, launches the kernel, and brings the answer back.

```
// 1. The kernel: runs ON THE GPU, once per thread. It is the CPU loop's body; each thread finds its own element.
__global__ void saxpyKernel(float a, const float* x, float* y, std::size_t n) {
    const std::size_t i = std::size_t(blockIdx.x) * blockDim.x + threadIdx.x;    // my block, my place in it: my element
    if (i < n) y[i] = a * x[i] + y[i];                                           // the same line as the CPU loop
}

// 2. The host function: runs ON THE CPU. This is what the rest of the program calls, and what the test times.
void saxpyG0(float a, const std::vector<float>& x, std::vector<float>& y) {
    const std::size_t N = x.size(), BYTES = N * sizeof(float);
    float* dX = nullptr; float* dY = nullptr;
    cudaMalloc(&dX, BYTES); cudaMalloc(&dY, BYTES);                            // GPU buffers, allocated on every call
    cudaMemcpy(dX, x.data(), BYTES, cudaMemcpyHostToDevice);                  // copy the inputs to the GPU
    cudaMemcpy(dY, y.data(), BYTES, cudaMemcpyHostToDevice);
    saxpyKernel<<<N, 1>>>(a, dX, dY, N);                                       // launch piece 1: N blocks of 1 thread each
    cudaMemcpy(y.data(), dY, BYTES, cudaMemcpyDeviceToHost);                 // copy the answer back
    cudaFree(dY); cudaFree(dX);                                                // and free, on every call
}
```

A first GPU port: allocate, copy in, launch, copy out, free. It gives the right answer; a unit test holds it to the CPU loop.

Simple example (GPU): the vernier test, and the commands

```
class Saxpy : public ::testing::TestWithParam<std::size_t> {}; // the parameter: how many elements

PERF_TEST_P(Saxpy, G0Naive) {
    PERF_GUARD(perf);
    const std::size_t N = GetParam();
    std::vector<float> x(N, 1.0f), y(N, 2.0f);
    perf.warmup([&] { saxpyG0(2.0f, x, y); });
    const auto R = perf.throughputLoop([&] { saxpyG0(2.0f, x, y); }, "saxpy_g0", N * sizeof(float));
    EXPECT_GT(R.callsPerSecond, 0.1);
}
```

The same shape as the CPU test. A second test, `SaxpyKernel`, times the kernel launch alone. Then the GPU tools, one flag each:

```
./Saxpy_PTEST --target-time 60ms --repeats 10 --csv sweep.csv           # time per call, every size
./Saxpy_PTEST --gtest_filter='SaxpyKernel.*' --csv kernel.csv          # the kernel alone, timed on the GPU
bench run Saxpy_PTEST --profile nsight -- --gtest_filter='*G0Naive/n1000000' # Nsight Systems
bench run Saxpy_PTEST --profile compute-sanitizer -- --gtest_filter='*G0Naive/n1000000' # GPU memory errors
sudo bench run Saxpy_PTEST --profile ncu -- --gtest_filter='SaxpyKernel.*' # Nsight Compute (needs root)
```

Simple example (GPU): the raw output

What those commands print for `saxpyG0` at 1 million elements, trimmed to the lines that matter. The next slide reads them.

```
the vernier test
[Length/Saxpy.G0Naive/n1000000] 3317.194 us/call CV=0.8% ~301 calls/s (p10=3272.244 p90=3334.389 sd=26.040)

--profile nsight
Time (%) Total Time (ns) Instances Avg (ns) Name
100.0 168,652,800 61 2,764,800.0 saxpyKernel(float, const float *, float *, unsigned long)
Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Name
63.4 177,205,921 183 968,338.4 60,102.0 cudaMemcpy
32.8 91,594,121 122 750,771.5 80,611.0 cudaMalloc
3.7 10,246,938 122 83,991.3 88,611.0 cudaFree

the SaxpyKernel test, kernel.csv GPU-timed kernel, CUPTI launch count
test,...,kernelTimeUs,transferTimeUs,...,cuptiKernelLaunches
SaxpyKernel.OneThreadPerBlock,...,2995.263367,69.328001,...,500

--profile ncu
saxpyKernel(float, const float *, float *, unsigned long) (1048576, 1, 1)x(1, 1, 1)
Memory Throughput % 4.79
Compute (SM) Throughput % 7.30
Achieved Occupancy % 29.10

--profile compute-sanitizer
===== ERROR SUMMARY: 0 errors
```

Simple example (GPU): the baseline results

One million elements (4 MB per array):

Reading	Tool	saxpyG0
Time per call	the vernier test	3,317 μs , rounds agree to 1%
Inside the kernel	<code>--profile nsight</code>	2,765 μs per launch: most of the call
In the three copies	<code>--profile nsight</code>	59 μ s
Allocating and freeing	<code>--profile nsight</code>	about 340 μ s: 2 <code>cudaMalloc</code> and 2 <code>cudaFree</code> per call
The kernel on its own	the vernier GPU test (CUDA events)	2,995 μ s per launch
How busy the GPU was	Nsight Compute	29% of its thread capacity
GPU memory errors	<code>--profile compute-sanitizer</code>	none

What it tells us: the copies aren't the problem. The kernel is slow while the GPU sits mostly idle, and a tenth of every call goes to allocating and freeing.

Simple example (GPU): the optimization

Two fixes, each one pointed at by a row of the last table:

```
class SaxpyG1 {                                // fix 1: the buffers outlive the call
public:
    explicit SaxpyG1(std::size_t n);            // cudaMalloc and cudaHostAlloc happen here, once
    void apply(float a, const std::vector<float>& x, std::vector<float>& y);
};

void SaxpyG1::apply(float a, const std::vector<float>& x, std::vector<float>& y) {
    std::memcpy(hX, x.data(), BYTES); std::memcpy(hY, y.data(), BYTES);        // into page-locked host buffers
    cudaMemcpyAsync(dX, hX, BYTES, cudaMemcpyHostToDevice, stream);
    cudaMemcpyAsync(dY, hY, BYTES, cudaMemcpyHostToDevice, stream);
    saxpyKernel<<<(N + 255) / 256, 256, 0, stream>>>(a, dX, dY, N);              // fix 2: blocks of 256 threads
    cudaMemcpyAsync(hY, dY, BYTES, cudaMemcpyDeviceToHost, stream);
    cudaStreamSynchronize(stream);                                                // one wait, at the end
    std::memcpy(y.data(), hY, BYTES);
}
```

- **Fix 1, allocate once:** no `cudaMalloc` or `cudaFree` in the call; the copies are queued, and the CPU waits once
- **Fix 2, fill the blocks:** the GPU schedules a block's threads 32 at a time. A block of 1 thread leaves 31 of every 32 slots empty; a block of 256 fills them

Simple example (GPU): the new results

Reading (1 million elements)	Tool	saxpyG0	SaxpyG1	
Time per call	the vernier test	3,317 μ s	610 μs	5.4× faster
Time inside the kernel	<code>--profile nsight</code>	2,765 μ s	22 μs	125× faster, same kernel
The kernel on its own	the vernier GPU test (CUDA events)	2,995 μ s	20 μ s	within 10% of Nsight, both times
API calls per saxpy call	<code>--profile nsight</code>	2 malloc, 2 free, 3 blocking copies	3 queued copies, 1 wait	no allocation
How busy the GPU was	Nsight Compute	29%	78%	the blocks are full
GPU memory errors	<code>--profile compute-sanitizer</code>	none	none	still correct
Where the time is now	<code>--profile gperf</code>		92% in <code>memcpy</code>	the kernel is 4% of the call

The kernel is the same source, character for character. The 5.4× came from how it is launched and fed, and each tool proved a different part of that: fewer API calls, fuller blocks, a faster kernel, still correct. The last row says where to look next.

A Real Application: The Workflow

What are we solving?

One function of x for every pair (n, m) . Each is defined from the two before it, scaled by two **coefficients**, A and B :

$$\bar{P}_{n,m}(x) = A_{n,m} \cdot x \cdot \bar{P}_{n-1,m}(x) - B_{n,m} \cdot \bar{P}_{n-2,m}(x)$$

$$A_{n,m} = \sqrt{\frac{(2n+1)(2n-1)}{(n-m)(n+m)}} \quad B_{n,m} = \sqrt{\frac{(2n+1)(n+m-1)(n-m-1)}{(2n-3)(n-m)(n+m)}}$$

the coefficients A and B depend only on n and m · $0 \leq m \leq n \leq N$, x in $[-1, 1]$

These are the **associated Legendre functions**, the core of spherical harmonics: how you describe anything on a sphere. Gravity, magnetic fields, weather, 3D audio, lighting.

The task: pick N , the highest degree you want (the level of detail), and a value x . Compute all of them. How fast can that be?

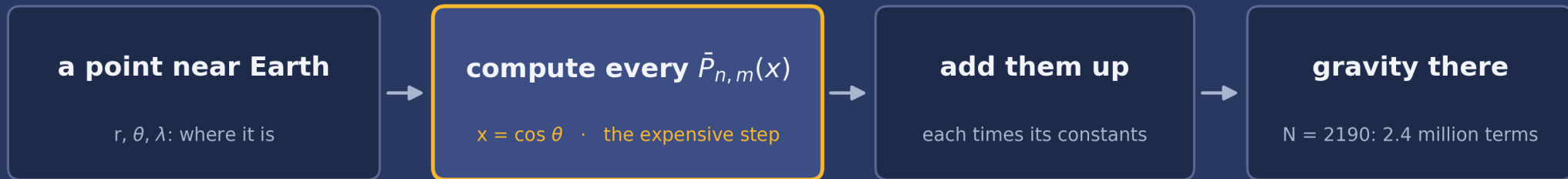
How is it used?

Why compute all of them? Because the value at one point needs every one.

Earth's gravitational potential at a point (the EGM2008 model):

$$V(r, \theta, \lambda) = \frac{GM}{r} \left\{ 1 + \sum_{n=2}^N \left(\frac{a}{r} \right)^n \sum_{m=0}^n (\bar{C}_{n,m} \cos m\lambda + \bar{S}_{n,m} \sin m\lambda) \bar{P}_{n,m}(\cos \theta) \right\}$$

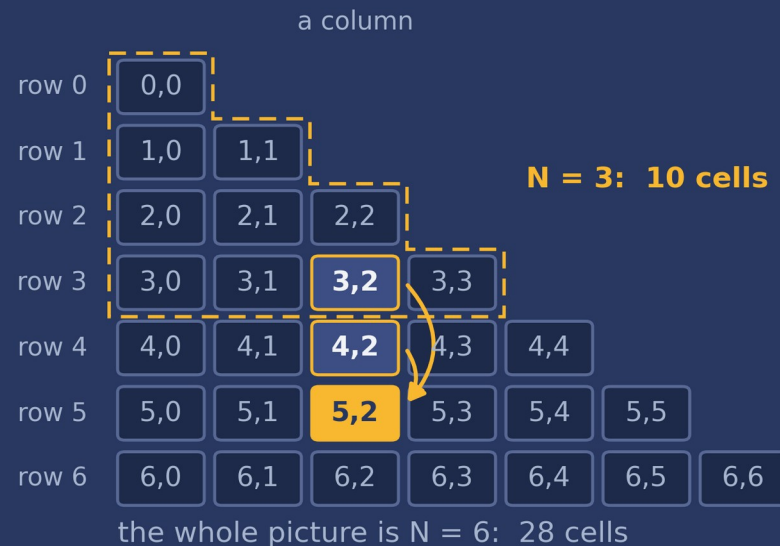
this example



A new point means a new x, and every function again. **This step has to be fast.**

What does that look like computationally?

A **triangular table** to fill: one cell for every pair (n, m) with $m \leq n$. And the equation says where each cell comes from:



N sets the size of the table.

$(N+1)(N+2)/2$ cells: rows 0 to N

**Each cell is computed from
the two cells above it in its column.**

A column is a chain; columns are independent.

The table grows with N: **N=50** is 1,326 cells, **N=180** is 16,471, **N=2190** is 2.4 million. In code, filling it is one function, `pbar(N, x, out)`. That's what we'll make fast.

The baseline code (CPU): one cell at a time

The function under test is pbar ("P-bar", the $\bar{P}$ from the equation): given N and x , fill the whole table. Everything we time and profile from here on is a call to it. We'll write it four ways, **pbarV0** to **pbarV3**.

V0 is the obvious one. A function that computes a **single cell** already exists (`std::sph_legendre`), so call it once per cell:

```
void pbarV0(int N, double x, double* out) {    // fills the table for one x; out has (N+1)(N+2)/2 slots
    for (int m = 0; m <= N; ++m)              // every column...
        for (int n = m; n <= N; ++n)          // ...every cell in it
            out[idx(n, m)] = K(m) * std::sph_legendre(n, m, theta); // one library call per cell
}
```

`idx(n, m)` is cell (n, m) 's slot in the array. `K(m)` and `theta = acos(x)` convert to the library's conventions.

Correct, and three lines of loop. **8,877 μ s per table at $N=180$** : that's the number every later version is measured against.

The CPU versions, and how each was found

The function under test is `pbar`: fill the whole table for one `x`. Four versions, each found by measuring the one before. Same answers every time: every cell is compared, version against version.

Version	What changed	Found with	What it showed	Time at N=180	vs V0
V0	One call per cell, to a function that computes a single cell.			8,877 μ s	1 $\times$
V1	Walked each column once and kept every value, instead of re-walking it for every cell.	<code>--profile callgrind</code> on V0	every one of those calls re-runs the same recurrence from the top of its column	93.4 μ s	95 $\times$
V2	Had the compiler compute the coefficients A and B (C++26 <code>constexpr</code>), instead of computing them for every cell on every call.	<code>--profile callgrind</code> on V1	46% of instructions on the two square-root lines	57.7 μ s	154 $\times$
V3	Replaced the $n(n+1)/2 + m$ index calculation with a running index.	<code>--profile callgrind</code> on V2	39% of instructions in the index arithmetic	35.4 μ s	251 $\times$

Most of the 251 $\times$ is the first step, a better algorithm. The profiler found the other two.

The GPU versions, and how each was found

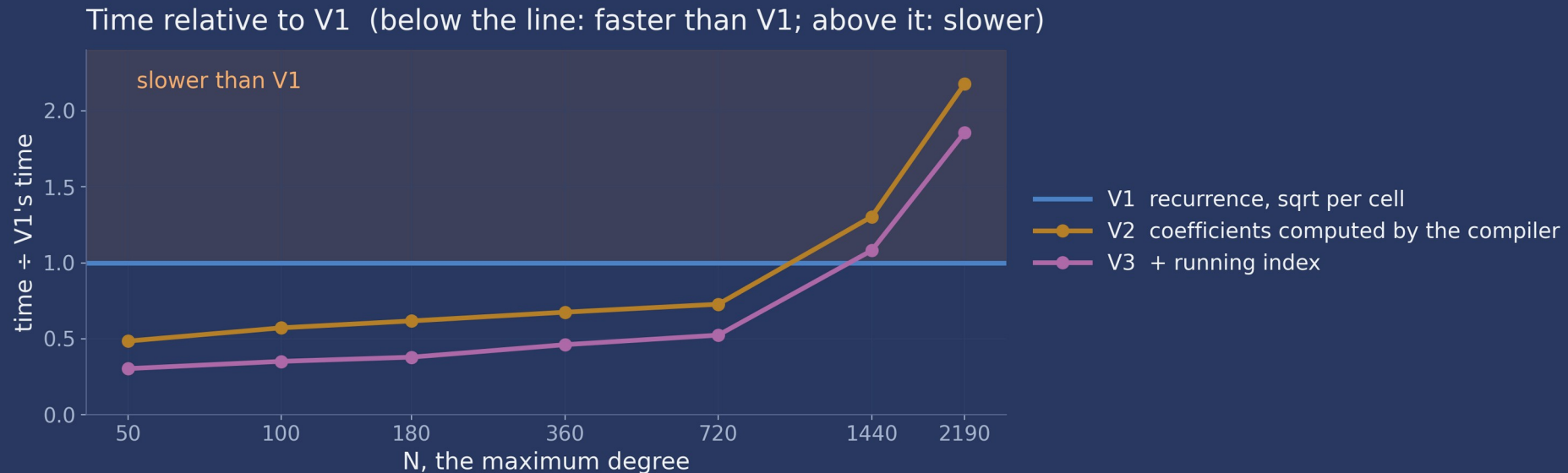
The same table on a GPU, for a batch of x values at once (G0 to G3, like the CPU's V0 to V3). A column is a chain, but columns, and different x values, are independent: that's what a GPU can use.

Step	What changed	Found with	What it showed	Per table, N=180, batch 32
G0 → G1	Allocated the GPU buffers once, not on every call.	--profile nsight (Nsight Systems)	3,904 <code>cudaMalloc</code> and 3,904 <code>cudaFree</code> calls for 61 batches	897 → 732 μs (1.2x)
G1 → G2	Launched the kernel once per batch , not once per x.	--profile nsight	1,952 launches of 722 μs. After: 61 launches of 1,038 μs, 32 tables each	732 → 40.0 μs (18x)
G2 → G3	Computed the square-root coefficients on the GPU once, not in every launch.	--profile ncu (Nsight Compute)	the kernel is compute-bound: 58% compute, 1.5% memory	40.0 → 15.6 μs (2.6x)

Kernel-only time, from vernier's own GPU kernel test with no external tool: 267 μs per launch of 32 tables.

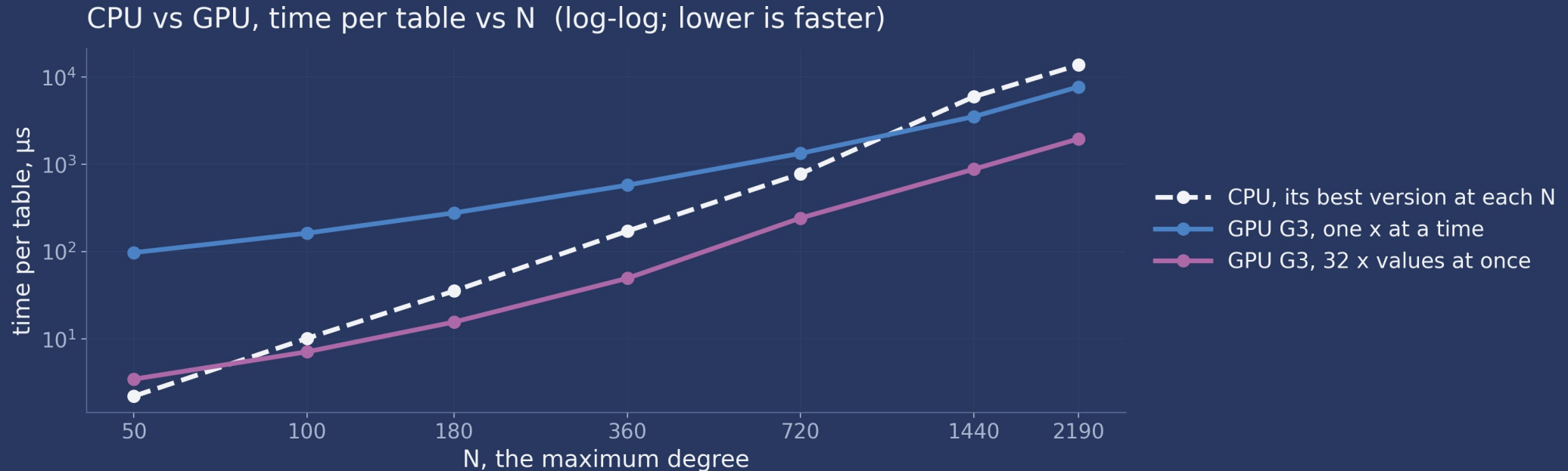
From the naive port to G3: **57x faster**. The kernel is the same source in all four; what changed is how much work each launch is given.

Niche topic 1: at high N, the cache decides



- ▶ Up to $N \approx 720$, V3 takes a third to a half of V1's time. **Past $N \approx 1000$ the order flips**: at $N=2190$, V1 takes 13.8 ms and V3 takes 25.6
- ▶ V3 reads three arrays per cell (the table, A and B); V1 reads one. At $N=180$ each is 132 KB and all three fit the core's 1 MB cache. At $N=2190$ each is **19 MB**
- ▶ So the fastest version is a **design decision that depends on your N**. A sweep shows that; a profile at one size can't

Niche topic 2: CPU vs GPU



- ▶ The CPU line is the **fastest CPU version at each N**: V3 up to N=720, V1 above it (the last slide)
- ▶ **One table at a time**, the GPU loses up to N=720 and then wins: 1.7× at N=1440, 1.8× at N=2190. A bigger table has more columns to run side by side
- ▶ **With a batch of 32** it wins from N=100 up: 1.4×, 2.3×, 3.5×, 3.2×, 6.8×, then **7× at N=2190**
- ▶ At N=50 the CPU wins either way: launching the GPU costs more than the work. **Small N: the CPU. Large N or many x values: the GPU, by up to 7×**

Key takeaways

- ▶ **A performance test can be an ordinary Google Test test.** Same file, same runner, same `ctest`. Vernier adds one object, `perf`, and the fixtures, parameters and filters you know keep working
- ▶ **Because the result is a value in the test, it can fail the build:** over a time budget, or too noisy to be evidence
- ▶ **Finding out why is one flag.** Fourteen profilers, CPU and GPU, run on that same test binary with `--profile`
- ▶ **Measure, ask a profiler, fix the one thing it points at, measure again.** That loop gave 38x on a string join, 5x on a GPU port, and 251x on a real application
- ▶ **Sweep the input; don't trust one size.** The fastest CPU version depended on N, and CPU against GPU depended on N and on the batch
- ▶ **Gate on real hardware, against a baseline from that same hardware.** A comparison fails the pull request. Once CI can reach the board, vernier needs nothing installed on it

Vernier is open source: github.com/apexedgesystems/vernier